

Estrutura de programa assembly



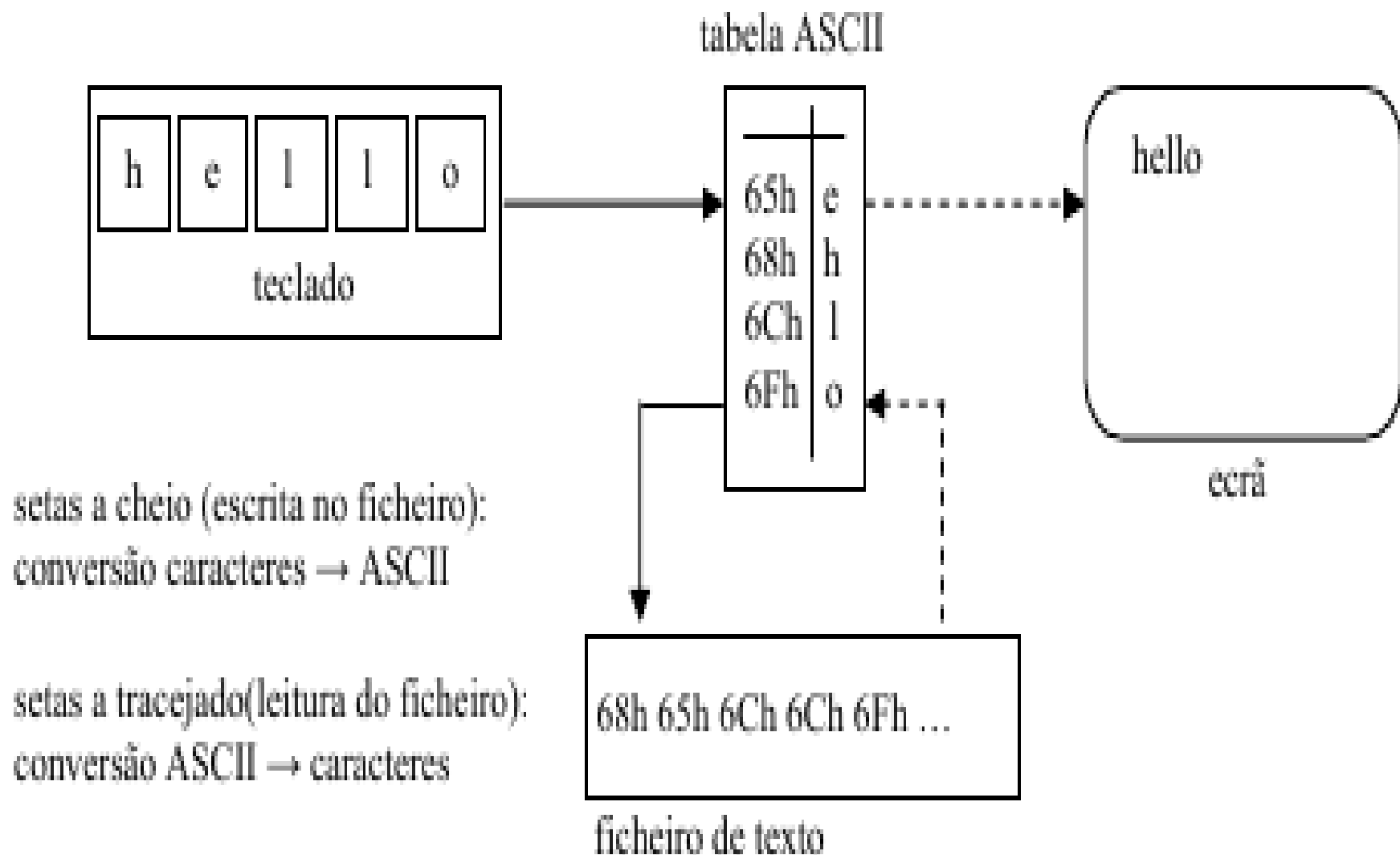
UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

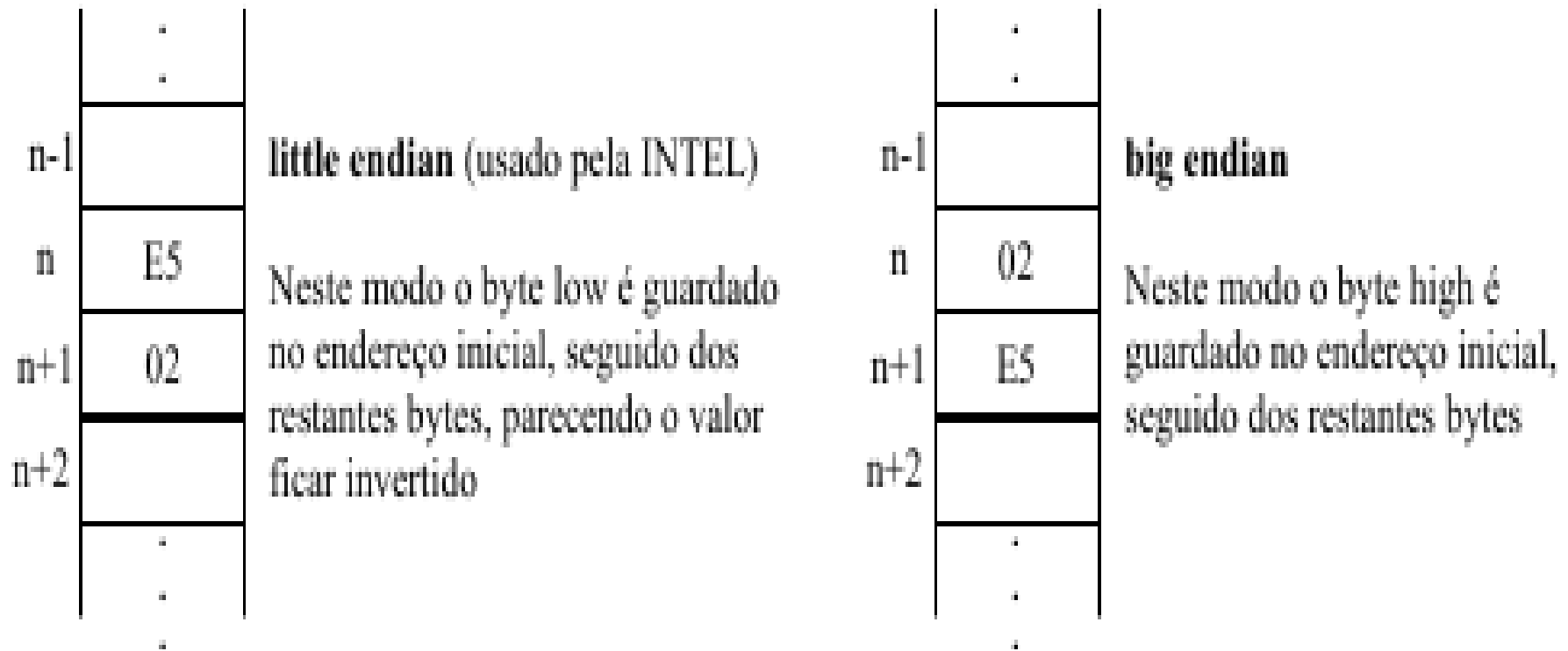
Prof. Carlos Minoru Tamaki

Armazenamento da informação



Organização de memória

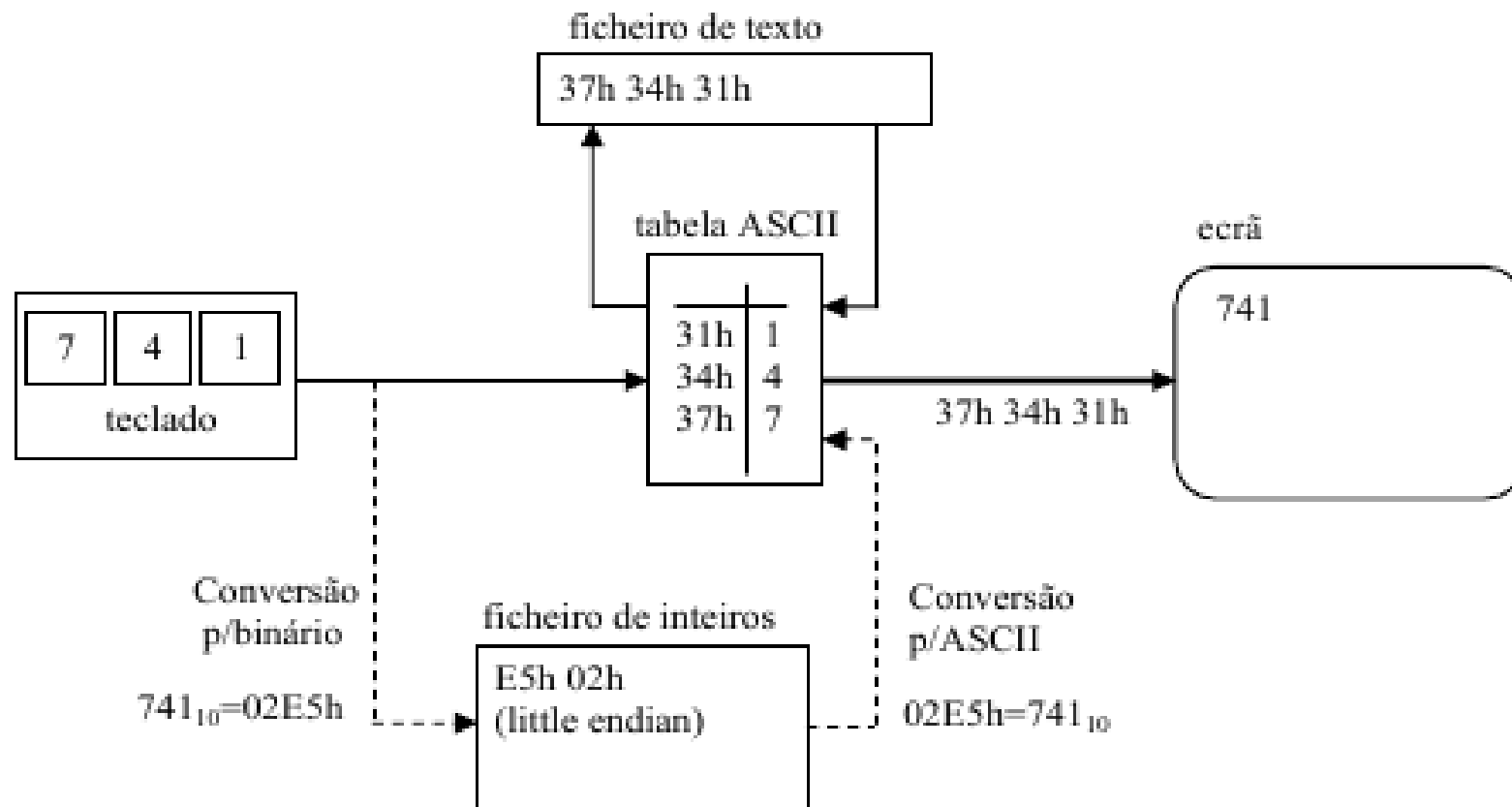
Memória organizada sequencialmente em bytes



Armazenamento do valor 741_{10} (0x02E5) na memória em computadores com organização diferente

Armazenamento de um valor numérico

O diagrama a seguir esquematiza o que acontece quando a sequência de teclas “741” é tratada como uma cadeia de caracteres (texto) ou como um valor numérico (inteiro).



setas a cheio: tratamento de texto (cadeias de caracteres)
setas a tracejado: tratamento de inteiros (valores numéricos)

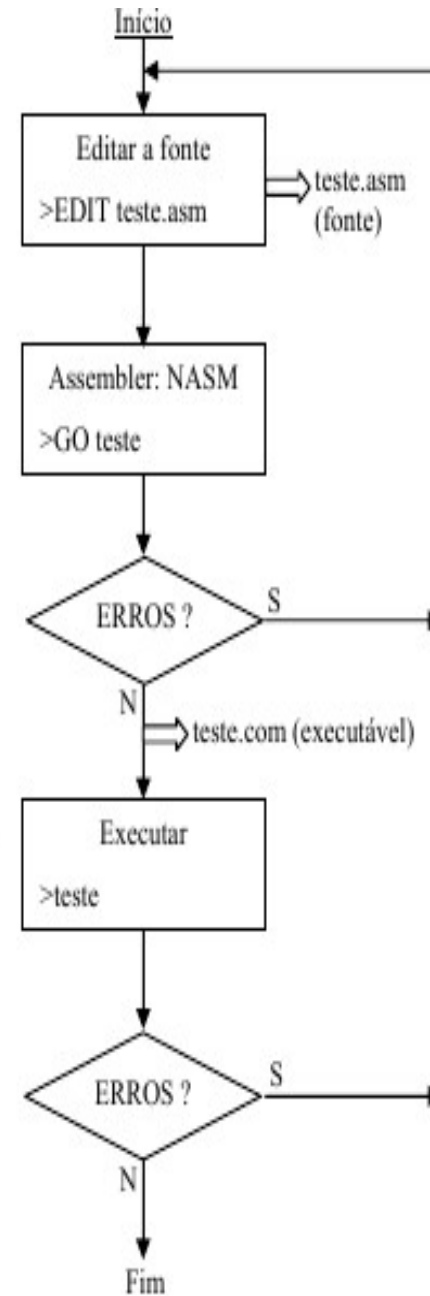
Escrita de programas

1º Passo: criar o programa fonte, usando um editor de texto; o ficheiro deverá ter extensão "asm"

2º Passo: converter o programa fonte para código-máquina usando o NASM (*)

(Verificar se há erros indicados pelo NASM; se houver, corrigi-los.)

3º Passo: executar o programa; verificar se o resultado é o esperado, caso contrário o programa ainda contém erros que devem ser corrigidos.



Estrutura de um programa NASM

section .data

;declaração e inicialização de dados

section .bss ; Block Started by Symbol

;declaração sem inicialização das informações que serão usadas

section .text ; Programa fonte

global _start

_start:

;o programa começa a partir daqui

Características principais do NASM

- **Comentários:** começam por “;” – tudo o que lhe seguir é ignorado pelo assembler. Podem ser aplicados a uma linha inteira ou apenas a parte. É boa ideia comentar as partes do programa cujo significado seja menos evidente (principalmente ao fim de algum tempo), como sejam os algoritmos utilizados, significado das variáveis, etc.
- **Maiúsculas/minúsculas:** o NASM é case-sensitive, ou seja, distingue entre elementos escritos em maiúsculas ou minúsculas. Esta regra não é universal, aplicando-se a nomes (constantes, variáveis), mas não a instruções, diretivas ou comentários. Por exemplo, a instrução “mov” pode ser escrita “Mov” ou “MOV”, pois é uma palavra reservada da linguagem, mas uma variável que tenha sido declarada com o nome “foo” não pode ser referenciada por “FOO”, caso contrário o NASM indica que a variável não existe.

- **Linha de código:** uma linha típica de código em NASM tem a forma:

<label>: **<instrução>** **<operandos>** ;comentário ← em que todos os elementos são opcionais i.e., uma linha de código pode não conter alguns deste elementos

<label>: indica um local para onde uma instrução de salto pode saltar (NOTA: a utilização dos dois pontos é opcional, mas é mais seguro utilizá-los. Se não forem usados o NASM pode considerar que uma instrução que foi escrita por engano é uma label , não indicando erro, mas fazendo com que o programa não trabalhe correctamente (ex. se numa linha com uma única instrução, escrever por engano “lodab” em vez da instrução correcta “lodsrb”, o NASM considera “lodab” uma label e não dá qualquer erro)

<instrução> uma das instruções (mnemônicas) do NASM (ex: mov, add, jmp)

<operandos> constantes, variáveis, registradores, etc, a que a instrução faz referência

Labels no nasm

- Para criar uma label, basta digitar o nome desse rótulo seguido de dois pontos, ' : '.
- Em seguida, escrevemos nossas linhas de código em Assembly.
- A sintaxe é a seguinte, então:

label:

```
;aqui vem o código  
;que será nomeado  
;de 'label'
```

- O NASM, que é o Assembler que estamos usando em nosso Curso de Assembly, permite a definição de labels locais.
- Para isso, basta colocar um ponto, ' . ', antes do nome da label.
- ***labelNormal*** -> label normal, sem ponto
- ***.labelInterna*** -> label local, com ponto, ela está dentro('pertence') a label 'labelNormal'
- Por exemplo, uma representação de um código que duas labels internas, de mesmo nome, mas em locais diferentes, que fazem a mesma coisa (incrementar), mas incrementam variáveis diferentes:

trecho1:

;trecho 1 do código

.incrementa:

inc numero1

;continuação trecho 1

trecho2:

;trecho 2 do código

.incrementa:

inc numero2

;continuação do trecho 2

Pseudo instruções

- **declaração de dados inicializados: declara dados com valor inicial**
 - **DB-define byte** → os valores são considerados bytes (8 bits)
 - db 0x55 (define o byte 0x55) , db 'a' (carácter 'a') , db 0 (inteiro 0) ,
 - db 255 (inteiro 255 – maior valor representado por um byte),
 - db 'hello',13,10 = db 'h','e','l','l','o',13,10 (define a string 'hello' seguida de CR/LF)
 - **DW-define word** → os valores são considerados words (16 bits)
 - dw 0x1234 (define uma word constituída pelos bytes 0x34 0x12)
 - dw 'a' (word 0x41 0x00) , dw 'ab' (word 0x41 0x42)
 - dw 65535 (inteiro 65535 – maior valor representado por uma word)
 - **DD-define double word** → os valores são considerados double-word (32 bits)
 - dd 0x12345678 (define uma double-word constituída pelos bytes 0x78 0x56 0x34 0x12)
 - dd 1.234567e20 (definição de uma constante em vírgula flutuante)

- **declaração de dados não inicializados: reserva espaço para armazenar valores**
 - **RESB-reserve byte** → buffer: resb 64 (reserva espaço para 64 bytes)
 - **RESW-reserve word** → wordvar: resw 1 (reserva espaço para uma word)
 - **RESD-reserve double word** → doublewordvar: resd 10 (array de 10 double-word)
- **comando EQU: atribui um valor a um símbolo (define uma constante)**
 - Ex: ecran **EQU** 1 ;define a constante “ecran” como sendo equivalente a 1

Referência a conteúdo/endereço de variáveis/memória

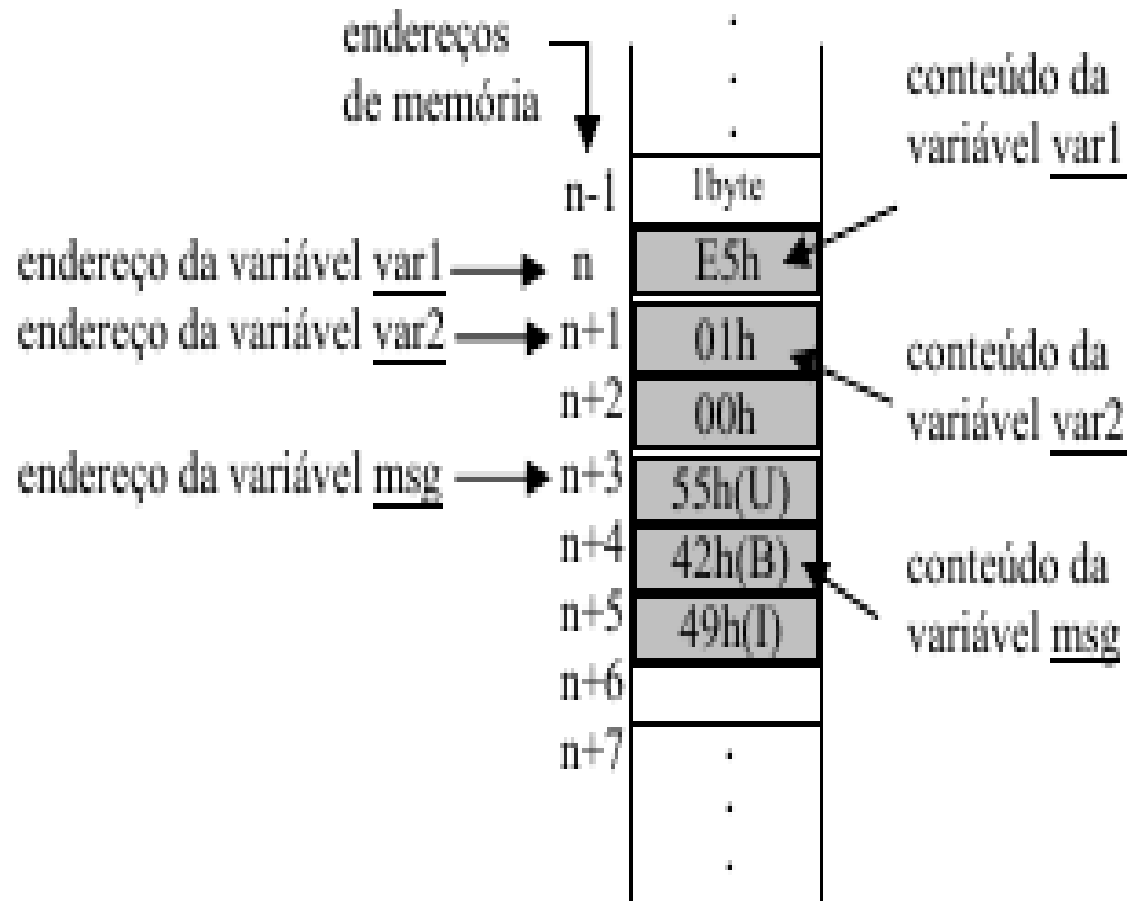
- O nome de uma variável representa o endereço de memória que foi atribuído a essa variável, mais propriamente o endereço do byte inicial dessa variável.

Ex: variáveis var1 tipo byte, var2 do tipo word (2 byte) e msg do tipo cadeia de caracteres:

var1 = E5h

var2 = 0001h

msg = 'UBI'



- **referências ao conteúdo de uma variável ou posição de memória, exigem que o endereço correspondente seja colocado entre colchetes “[]” ;**
 - ex: mov ax, [ind1] ;move o conteúdo da variável ind1 para o registrador ax
- **referências ao endereço das variáveis (i.e., à sua posição na memória) não levam colchetes**
 - ex: mov dx, ind2;move o endereço da variável ind2 para o registrador dx
- **NOTA:** não são permitidas referências à memória/variáveis para origem e destino de dados dentro da mesma instrução.
 - ex: mov [ind2] , [ind1]

- **ERRO:** não é possível mover o conteúdo de uma variável(memória) diretamente para outra variável(memória); então devemos escrever da seguinte forma:
 - mov ax, [ind1] ; usa-se um registrador auxiliar (neste caso o ax), para
 - mov [ind2], ax ;permitir a operação
- **endereços efetivos:** qualquer operando de uma instrução que faz referência à memória.
 - Exs:
 - mov al, [msg] ;coloca no registrador al o 1o byte do conteúdo da variável msg (al 55h='U')
 - mov ah, [msg+1] ;coloca no registrador ah o 2o byte do conteúdo da variável msg (ah 42h='B')
 - mov bl, [msg+2] ;coloca no registrador ah o 3o byte do conteúdo da variável msg (bl 49h='I')

Outros exemplos:

- **mov si, msg** ;coloca no registrador si, o endereço da variável msg (si n+6)
- **mov al, [si]** ;coloca no registrador al, o conteúdo da posição de memória ;apontada pelo registrador si, ou seja, o carácter 'U' (al 55h='U')
- **inc si** ;incrementa de uma unidade o registrador si (si n+6+1)
- **mov al, [si]** ;coloca no registrador al, o conteúdo da posição de memória ;apontada pelo registrador si, ou seja o carácter 'B' (al 42h='B')
- **inc si** ;incrementa de uma unidade o registrador si (si n+6+1+1)
- **mov al, [si]** ;coloca no registrador al, o conteúdo da posição de memória ;apontada pelo registrador si, ou seja o carácter 'I' (al 49h='I')

- **O NASM não memoriza os tipos das variáveis:** quando se declaram variáveis usando as pseudo-instruções para dados inicializados ou não-inicializados, o NASM apenas memoriza o endereço de memória que foi atribuído à variável (para lhe poder aceder), mas “esquece” imediatamente o tipo dessa variável. Isto implica que o NASM obriga a que se indique o tipo de uma variável sempre que esta é referida.
- Ex1:
- `bytevar: resb 1` ;declara a variável “bytevar” como sendo um byte (8 bit)
- `mov [bytevar],10` ;provoca erro, pois o NASM esqueceu o tipo de “bytevar”, não conseguindo atribuir-lhe o valor 10
- `mov byte [bytevar],10` ;assim o NASM já consegue atribuir o valor à ;variável

- **Ex2:**

- `wordvar: resw 1` ;declara a variável “wordvar” como sendo uma; (16 bit)
- `mov [wordvar], 100` ;provoca erro, pois o NASM esqueceu o tipo de ;“wordvar”, não conseguindo atribuir-lhe o valor 100
- `mov word [wordvar],100` ;assim o NASM já consegue atribuir o valor à ;variável

- **Ex3: quando as expressões envolvem registradores não é preciso indicar tipos**

- `mov al,10` ;não há erro, pois o NASM “sabe” que o tipo do registrador é byte
- `mov cx,100` ;não há erro, pois o NASM “sabe” que o tipo do ;registrador cx é word

Estruturas de controle

- As linguagens de alto nível possuem estruturas de controle sequencial, condicional e repetição
- Muitas linguagens de montagem não possuem essas estruturas e é necessário montar estas estruturas com instruções simples

Estrutura Sequencial

- São estruturas onde as instruções são executadas na sequência em que foram escritas
- Exemplo:

Estrutura Sequencial

- São estruturas onde as instruções são executadas na sequência em que foram escritas
- Exemplo:

```
section .text
global _start
_start:
    mov     edx,len
    mov     ecx,msg
    mov     ebx,1
    mov     eax,4
    int     0x80      ;interrupção (trap)
    mov     eax,1
    int     0x80
```

```
section .data
msg      db  'Hello, world!',0xa
len      equ  $ - msg
```

Estrutura de Seleção - Se-então

- Estrutura que auxilia na seleção de alguma ação
- Estrutura Se-então (if-then)

Se (condição)

Então

Instrução 1

Instrução 2

Instrução 3

Fim-se

Estrutura de Seleção - Se-então

- Estrutura que auxilia na seleção de alguma ação
- Estrutura Se-então (if-then)

Se (condição)

Então

Instrução 1

Instrução 2

Instrução 3

Fim-se

Instrução que afeta flags

Salta se não condição para Fim-se

Instrução 1

Instrução 2

Instrução 3

Fim-se

Exemplo - Se-então

Se ($a > 10$)

Então

$b = a - 5;$

$c = x + b;$

Fim-se

Exemplo - Se-então

Se (a > 10)

Então

$b = a - 5;$

$c = x + b;$

Fim-se

cmp [a], 10

jle fim_se ;jump less equal

mov ax, [a]

sub ax, 5

mov [b], ax ; ax = b

add ax, [x]

mov [c], ax

fim_se:

Exemplo - Se-então

Se (a > 10)

Então

b = a - 5;

c = x + b;

Fim-se

cmp [a], 10

jle fim_se ;jump less equal

mov ax, [a]

sub ax, 5

mov [b], ax ; ax = b

add ax, [x]

mov [c], ax

fim_se:

Jle – Jump Less Equal – Salte se menor ou igual, flags Zero = 1 e Sinal $\neq$ Overflow

Estrutura de Seleção - Se-então-senão

Estrutura Se-então-senão (if-then-else)

Se (condição)

Então

 Instrução 1

 Instrução 2

Senão

 Instrução 3

 Instrução 4

Fim-se

Estrutura de Seleção - Se-então-senão

Estrutura Se-então-senão (if-then-else)

Se (condição)

Então

 Instrução 1

 Instrução 2

Senão

 Instrução 3

 Instrução 4

Fim-se

Instrução que afeta flags

Salta se não condição para Senão

 Instrução 1

 Instrução 2

 Salta para Fim-se

Senão

 Instrução 3

 Instrução 4

Fim-se

Exemplo - Se-então-senão

Se ($a > 10$)

Então

$b = a - 5;$

$c = x + b;$

Senão

$b = a + 3;$

$c = y - b;$

Fim-se

Exemplo - Se-então-senão

Se (a > 10)

Então

$b = a - 5;$

$c = x + b;$

Senão

$b = a + 3;$

$c = y - b;$

Fim-se

```
cmp [ a ], 10
```

```
jle else ;jump less equal
```

```
mov ax, [ a ]
```

```
sub ax, 5
```

```
mov [ b ], ax
```

```
add ax, [ x ]
```

```
mov [ c ], ax
```

```
jmp fim_se
```

```
else:
```

```
mov ax, [ a ]
```

```
add ax, 3
```

```
mov [ b ], ax
```

```
mov ax, [ y ]
```

```
sub ax, [ b ]
```

```
mov [ c ], ax
```

```
fim_se:
```

Estruturas de Repetição - Enquanto-faça

São estruturas utilizadas para realizar ações repetidas

Estrutura Enquanto-faça (while-do)

Enquanto (condição)
faça

 Instrução 1

 Instrução 2

Fim-enquanto

Estruturas de Repetição - Enquanto-faça

São estruturas utilizadas para realizar ações repetidas

Estrutura Enquanto-faça (while-do)

Enquanto (condição)
faça

 Instrução 1

 Instrução 2

Fim-enquanto

Enquanto:

 Instrução que afeta flags

 Salta se não condição para Fim-enquanto

 Instrução 1

 Instrução 2

 Salta para enquanto

Fim-enquanto

Exemplo - Enquanto-faça

`a = 10;`

`Enquanto (a >
0) faça`

`b = a - 5;`

`c = x + b;`

`a = a - 1;`

`Fim-enquanto;`

Exemplo - Enquanto-faça

a = 10;

Enquanto (a > 0) faça

 b = a – 5;

 c = x + b;

 a = a – 1;

Fim-enquanto;

mov [a], 10

Enquanto:

 cmp [a], 0

 jle Fim_enquanto ;jump less equal

 mov ax, [a]

 sub ax, 5

 mov [b], ax

 add ax, [x]

 mov [c], ax

 dec [a]

jmp Enquanto

Fim_enquanto:

Estrutura Faça-enquanto (do-while)

Faça

 Instrução 1

 Instrução 2

Enquanto (condição)

Estrutura Faça-enquanto (do-while)

Faça

 Instrução 1

 Instrução 2

Enquanto (condição)

Faça:

 Instrução 1

 Instrução 2

Instrução que afeta
flags

Salta se condição
para Faça

Exemplo Faça-enquanto (do-while)

`a = 10;`

Faça

`b = a - 5;`

`c = x + b;`

`a = a - 1;`

Enquanto ($a > 0$)

Exemplo Faça-enquanto (do-while)

a = 10;

Faça

b = a - 5;

c = x + b;

a = a - 1;

Enquanto (a > 0)

mov [a], 10

Faça:

mov ax, [a]

sub ax, 5

mov [b], ax

add ax, [x]

mov [c], ax

dec [a]

cmp [a], 0

jgt Faça ;jump greather than

Estrutura Para-faça (for-do)

Para $i = A$ até B faça

 Instrução 1

 Instrução 2

Fim-para

Estrutura Para-faça (for-do)

Para $i = A$ até B faça

 Instrução 1

 Instrução 2

Fim-para

$i = A;$

Para:

 Instrução 1

 Instrução 2

 Incremente i

 Compara i com B

 Se $i \leq B$ salta para Para

Fim-para:

Exemplo Para-faça (for-do)

Para $i = 1$ até 10

$b = a - 5;$

$c = x + b;$

Fim-para

Exemplo Para-faça (for-do)

Para i = 1 até 10

$b = a - 5;$

$c = x + b;$

Fim-para

mov [i], 1

Para:

mov ax, [a]

sub ax, 5

mov [b], ax

add ax, [x]

mov [c], ax

inc [i]

cmp [i], 10

jle Para ;jump less equal

Fim-para: ; label opcional

Bibliografia

- Organização Estruturada de Computadores
– Andrew S. Tanenbaum, Tood Austin – 6ª
Edição 2013 Prentice Hall
- Vários sites da internet